

SmartPost AI

AI-Based Automated Social Media Content
Generation and Publishing System

Project Report

Team Members

Peter Barakat Abboud — 119894 — Fr
Toufik Amine Dawaliby — 120279 — En

June 2026

Table of Contents

Table of Contents.....	2
1. Introduction	5
2. Project Objectives	5
3. Core Features (Implemented).....	5
3.1 User Authentication & Registration.....	5
3.2 Social Media Account Integration (LinkedIn).....	5
3.3 AI Research-Based Post Generation	6
3.4 Post Management.....	6
3.5 Scheduled Auto-Posting.....	6
3.6 Dashboard.....	7
4. Technology Stack	7
5. System Architecture	8
5.1 Architecture Layers.....	8
5.2 API Endpoint Summary	9
6. Database Design	9
6.1 Entity Descriptions	9
6.2 Entity Relationships	10
7. Frontend Architecture	10
7.1 Component Structure	10
7.2 Key Design Patterns	11
8. Python Scraping & AI Modules	11
9. Background Job Processing.....	12
10. Security Considerations	12
11. Application Flows.....	13
11.1 Signup & Email Verification Flow.....	13
11.2 AI Post Generation Flow	13
11.3 Scheduled Posting Flow	14
12. Project Structure.....	14
13. Installation & Deployment.....	15
13.1 Prerequisites	15
13.2 Installation Steps	15
13.3 Running the Application	15
14. Diagrams	15
14.1 Use Case Diagram	15
14.2 Class Diagram.....	16

14.3 Sequence Diagram	17
14.4 Flowchart	19
15. Appendix	22
A.1 System Architecture	22
A.1.1 Overall Architecture Explanation	23
A.1.2 Component Interactions.....	23
A.1.3 Data Flow Descriptions.....	24
A.1.4 Client-Server Communication	25
A.2 Database Design	26
A.2.1 Entity Descriptions.....	26
A.2.2 Relationships Between Tables.....	26
A.2.3 Database Schema Explanations.....	27
A.2.4 Sample Records and Field Descriptions.....	27
A.3 API Documentation	29
A.3.1 Authentication Mechanisms	29
A.3.2 Endpoint Descriptions	29
A.3.3 Request and Response Examples	30
A.3.4 Error Handling and Status Codes.....	31
A.4 User Interface Screens.....	32
A.4.1 Page Descriptions and Features	32
A.4.2 Buttons, Controls, and Input Forms	32
A.4.3 User Workflows and Navigation.....	32
A.5 Implementation Details.....	34
A.5.1 Frontend Technologies and Libraries	34
A.5.2 Backend Architecture and Packages	34
A.5.3 State Management and Hooks.....	34
A.5.4 External Integrations and APIs	34
A.6 LinkedIn Integration	37
A.6.1 OAuth 2.0 Authentication Process	37
A.6.2 Access Token Management and Lifetime.....	37
A.6.3 Post Creation Workflow	37
A.6.4 Image Upload and Binary Transmission	37
A.6.5 Error Handling and Security Considerations	38
A.7 AI Content Generation Module	39
A.7.1 Prompt Generation Process	39
A.7.2 Workflow and Processing Steps	39
A.7.3 Integration with External AI Services	39
A.7.4 Response Handling and JSON Parsing	39

A.8 Redis and Caching Mechanisms	41
A.8.1 Purpose of Redis in SmartPost	41
A.8.2 Data Storage Strategy and Configurations	41
A.8.3 Queue Handling and Background Workers	41
A.8.4 Session Management and System Reliability	41
A.9 n8n Automation Workflows	42
A.9.1 Workflow Descriptions	42
A.9.2 Trigger Mechanisms	42
A.9.3 Data Processing Stages	42
A.9.4 Integration with Third-Party Services.....	42
A.10 Security Measures	43
A.10.1 Authentication and Authorization.....	43
A.10.2 Environment Variables	43
A.10.3 Token Protection	43
A.10.4 Input Validation	43
A.10.5 Rate Limiting and Error Management.....	43
A.11 Testing and Validation	45
A.11.1 Unit Testing	45
A.11.2 Integration Testing	45
A.11.3 Functional Testing	45
A.11.4 Test Scenarios and Expected Results.....	45
A.12 Deployment Process.....	47
A.12.1 Railway Deployment Architecture.....	47
A.12.2 Environment Configuration	47
A.12.3 Production Setup.....	47
A.12.4 CI/CD Pipeline	47
A.13 Sample Code Listings	48
16. Challenges & Risks	52
17. Future Improvements	52
18. Conclusion.....	52

1. Introduction

SmartPost AI is a full-stack web application that enables users to generate, manage, and automatically publish social media content using Artificial Intelligence. Users can connect their social media accounts (currently LinkedIn), generate AI-powered posts based on research from YouTube and other sources, and schedule automated, recurring content publishing across platforms.

The application combines AI-powered content generation, scraping-based research, scheduling automation, and multi-platform publishing into a unified dashboard experience.

2. Project Objectives

- Provide users with an AI-driven tool to create high-quality social media content from user-defined topics.
- Automate the research phase by scraping YouTube videos, comments, Google search results, Facebook posts, and Telegram channels.
- Enable direct publishing to connected social media platforms (LinkedIn), with full CRUD operations on posts.
- Support scheduled/recurring content generation and auto-publishing via a background job queue.
- Ensure secure user authentication with email verification (OTP), JWT tokens, and Google OAuth2 sign-in.
- Provide an intuitive, modern, and responsive user interface with dark/light theme support.

3. Core Features (Implemented)

3.1 User Authentication & Registration

- User registration with email and password.
- Email verification via a 6-digit OTP code sent through Brevo SMTP API.
- OTP expires after 15 minutes; limited to 5 verification attempts.
- Resend OTP functionality with rate limiting (max 3 resends per hour per IP).
- Secure password hashing using bcrypt (10 salt rounds).
- JWT-based authentication with 7-day token expiry.
- Google OAuth2 sign-in using the google-auth-library — new Google users must also verify their email before account creation.
- Role system: 'standard' and 'pro' user roles stored in the database.

3.2 Social Media Account Integration (LinkedIn)

The application currently integrates with LinkedIn as the primary social media platform:

- OAuth 2.0 authorization flow — user is redirected to LinkedIn's consent page.
- Authorization code exchange for access + refresh tokens.

- Tokens stored in the database with expiry tracking.
- Automatic token refresh when an access token is within 60 seconds of expiry.
- Profile data retrieval (name, URN) via the LinkedIn OpenID /v2/userinfo endpoint.
- Account disconnect functionality — removes the SocialAccount record from the database.
- LinkedIn API versioning header (202603) for forward compatibility.

3.3 AI Research-Based Post Generation

The content generation pipeline follows these steps:

- **Step 1 — Topic Input:** The user provides a text prompt describing their desired topic and selects a tone/attitude (Neutral, Professional, Casual, or Viral).
- **Step 2 — Web Scraping:** The YouTube scraper (Python, using the YouTube Data API v3) searches for videos matching the prompt, fetches full descriptions, view/like/comment statistics, and top comments for each video.
- **Step 3 — AI Post Generation:** The scraped data is sent as JSON to a Google Scraper webhook (n8n automation) that uses an LLM to generate an analytical report and a publish-ready social media post.
- **Step 4 — Image Generation (Optional):** If requested, an AI image is generated via the Pollinations AI API (flux model), saved to the server's public directory, and served statically.
- **Step 5 — Database Persistence:** The generated post is saved as a 'draft' in the Post table, and the scraped research data is saved as a ResearchReport linked to that post.

Additional scrapers exist in the codebase for Facebook (Playwright-based, headless Chrome), Google (Playwright with AI Mode detection), and Telegram (Telethon client) — these can be activated as needed.

3.4 Post Management

- Create new posts and save as drafts.
- Publish draft posts directly to LinkedIn.
- Edit post content (updates both the local DB and the LinkedIn post via PATCH with X-HTTP-Method-Override).
- Delete posts (soft-delete locally + remote delete via LinkedIn REST API).
- View all non-deleted posts with status, content, and creation date.
- Copy generated post text to clipboard.
- Content moderation: a configurable list of banned words prevents inappropriate prompts from being processed.

3.5 Scheduled Auto-Posting

Users can define recurring content generation schedules:

- Set a topic/prompt, select tone, choose a repeat interval (from every 2 minutes to weekly), and pick a start date/time.
- Optionally enable auto-publish to LinkedIn for each generated post.
- Background processing uses BullMQ (Redis-backed job queue): each job runs the full scrape → generate → save pipeline.

- After each successful execution, the worker automatically re-queues the next job based on the configured interval.
- Email notifications are sent to the user after each execution — success or failure — via the Brevo API.
- Users can view all active schedules and cancel them (which also removes waiting/delayed BullMQ jobs).
- If a job fails, the scheduled task is automatically deactivated and an error email is sent.

3.6 Dashboard

The dashboard provides a centralized landing page with:

- Personalized welcome greeting based on the logged-in user's email.
- Quick-access cards to the three main features: AI Post Generator, Post Scheduler, and LinkedIn management.
- Stats strip displaying platform capabilities (AI models, platforms, post tones, schedule intervals).
- Quick tips section to guide new users.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React 19.2 + Vite 7.3	Single-page application with fast HMR development
Frontend	React Router DOM 7.13	Client-side routing and navigation guards
Frontend	Axios 1.13	HTTP client for API communication
Frontend	Lucide React 1.11	Icon library for modern UI icons
Frontend	@react-oauth/google 0.13	Google One Tap / OAuth2 sign-in component
Frontend	Vanilla CSS	Custom styling with dark/light theme support via CSS custom properties
Backend	Node.js + Express 5.2	REST API server
Backend	Prisma 6.19	ORM for MySQL database access and schema migrations
Backend	bcrypt 6.0	Password hashing
Backend	jsonwebtoken 9.0	JWT token generation and verification
Backend	BullMQ 5.71 + ioredis 5.10	Redis-backed background job queue for scheduled post generation
Backend	Nodemailer 8.0 / Brevo API	Email sending for verification codes and notifications
Backend	google-auth-library 10.6	Google OAuth2 token verification on the server

Layer	Technology	Purpose
Backend	express-rate-limit 8.2	Rate limiting for sensitive endpoints (e.g., OTP resend)
Python	YouTube Data API v3 (google-api-python-client)	YouTube search, video metadata, and comment scraping
Python	Playwright	Browser automation for Google and Facebook scraping
Python	Telethon	Telegram channel message scraping
Python	Requests + Pollinations AI	AI image generation
Database	MySQL 8.0	Relational database for all persistent data
Infrastructure	Redis	Message broker for BullMQ job queue
Automation	n8n (webhook)	LLM orchestration — receives scraped data and generates post text

5. System Architecture

SmartPost AI follows a layered architecture with clear separation of concerns:

User → React Frontend (Vite) → Express REST API (Node.js) → Service Layer → MySQL Database

5.1 Architecture Layers

- **Presentation Layer (Frontend):** A React SPA served by Vite, communicating with the backend via Axios HTTP calls. Handles routing, auth guards, theme toggling, and all UI rendering.
- **API Layer (Express Routes):** RESTful endpoints organized by domain — auth, users, generate, scheduler, linkedin, health. Each route module applies the JWT authentication middleware where needed.
- **Business Logic (Controllers):** Four controllers (auth, generate, linkedin, scheduler) encapsulate all domain logic, calling Prisma for database operations and spawning Python child processes for scraping.
- **Background Processing (BullMQ Worker):** A Redis-backed job queue processes scheduled post generation asynchronously. The worker runs the full pipeline (scrape → LLM → save → optionally publish) and re-queues itself for the next interval.
- **Data Access (Prisma ORM):** All database interactions go through Prisma Client, providing type-safe queries, schema migrations, and relation management.
- **External Services:** YouTube Data API, Pollinations AI, Brevo SMTP, LinkedIn REST API, and an n8n webhook for LLM post generation.

5.2 API Endpoint Summary

Method	Endpoint	Auth	Description
POST	/api/auth/signup	No	Register with email/password — sends OTP
POST	/api/auth/verify-email	No	Verify OTP and create user account
POST	/api/auth/resend-code	No	Resend OTP (rate-limited: 3/hour)
POST	/api/auth/login	No	Login and receive JWT
POST	/api/auth/google	No	Google OAuth2 sign-in/sign-up
POST	/api/generate	Yes	Generate AI post from prompt
GET	/api/linkedin/login	Yes	Initiate LinkedIn OAuth2 flow
GET	/api/linkedin/callback	No	LinkedIn OAuth2 callback
GET	/api/linkedin/status	Yes	Check LinkedIn connection status
GET	/api/linkedin/posts	Yes	List all LinkedIn posts
POST	/api/linkedin/post	Yes	Publish text post to LinkedIn
PATCH	/api/linkedin/post/:postId	Yes	Edit LinkedIn post
DELETE	/api/linkedin/post/:postId	Yes	Delete LinkedIn post
DELETE	/api/linkedin/disconnect	Yes	Disconnect LinkedIn account
POST	/api/scheduler	Yes	Create scheduled task
GET	/api/scheduler	Yes	List active scheduled tasks
DELETE	/api/scheduler/:id	Yes	Cancel a scheduled task
GET	/api/health	No	Health check endpoint

6. Database Design

The database uses MySQL 8.0 with Prisma ORM for schema management and migrations. The schema defines 7 models (tables) with the following relationships:

6.1 Entity Descriptions

Entity	Key Fields	Description
User	id, email, password, role, createdAt	Stores registered users. Password is bcrypt-hashed. Role is 'standard' or 'pro'.

Entity	Key Fields	Description
Platform	id, platformName, apiBaseUrl, isActive	Represents a social media platform (e.g., LinkedIn, Smartpost Default).
SocialAccount	id, userId, platformId, accessToken, refreshToken, expiresAt, linkedinUrn, linkedinName	Links a user to a platform with OAuth tokens. Unique constraint on (userId, platformId).
Post	id, socialAccountId, content, mediaUrl, type, status, externalPostId, deleted, scheduledAt, createdAt	Individual posts. Type: text/image/video. Status: draft/scheduled/posted/failed.
ResearchReport	id, postId, topic, generatedReport, createdAt	Stores the raw scraped research data (JSON) associated with a generated post.
ScheduledTask	id, topic, intervalHours, nextExecution, isActive, publishLinkedIn, socialAccountId, createdAt, updatedAt	Defines a recurring auto-generation task with interval and auto-publish settings.
EmailVerification	id, email, password, code, expiresAt, attempts, createdAt	Temporary storage for pending email verifications (deleted after successful verification).

6.2 Entity Relationships

- User 1 → N SocialAccount (a user can connect multiple platform accounts)
- Platform 1 → N SocialAccount (a platform can have many linked accounts)
- SocialAccount 1 → N Post (each post belongs to one social account)
- SocialAccount 1 → N ScheduledTask (scheduled tasks are tied to a social account)
- Post 1 → N ResearchReport (each post can have associated research data)
- Unique constraint: (userId, platformId) on SocialAccount — one account per platform per user

Performance indexes are defined on: Post.status, Post.scheduledAt, ScheduledTask.nextExecution, and ScheduledTask.isActive.

7. Frontend Architecture

7.1 Component Structure

Component	Route	Description
Login	/login	Email/password login form with Google OAuth2 sign-in button
Signup	/signup	Registration form that triggers OTP email verification

Component	Route	Description
EmailVerify	/verify-email	6-digit OTP entry with resend functionality
Dashboard	/	Landing page with feature cards, stats, and quick tips
Generate	/generate	AI post generation form with attitude selection, image toggle, and result display
Scheduler	/schedule	Create recurring schedules and view/cancel active tasks
LinkedIn	/linkedin	Connect/disconnect LinkedIn, view posts, create/edit/delete posts
Navbar	(layout)	Navigation bar with links, theme toggle, and logout

7.2 Key Design Patterns

- **Auth Guard (RequireAuth):** A wrapper component that checks for a JWT token in localStorage. Unauthenticated users are redirected to /login.
- **Theme System:** A ThemeContext provides dark/light mode toggling. The theme is persisted in localStorage and applied via a data-theme attribute on <html>, with CSS custom properties adapting colors.
- **Layout Pattern:** Auth pages (Login, Signup, EmailVerify) render without the Navbar. All other pages are wrapped in an AppLayout component that includes the Navbar.
- **API Communication:** All API calls use Axios with the VITE_API_URL environment variable. JWT tokens are attached via the Authorization: Bearer header.

8. Python Scraping & AI Modules

The backend delegates scraping and image generation to Python scripts invoked as child processes via Node.js's spawn(). Each script outputs a JSON result to stdout.

Script	Technology	Description
youtube_scraper.py	YouTube Data API v3	Searches for videos by prompt, fetches metadata + statistics, and extracts top comments. Returns structured JSON with video details and comment text.
google_scrape.py	Playwright (Chrome)	Opens Google, searches for public discussions on the topic, attempts Google AI Mode, and extracts text snippets.
facebook_scrape.py	Playwright (Chrome)	Uses a persistent Chrome session to search Facebook posts, scrolls,

Script	Technology	Description
		clicks 'See more', and collects post text.
telegram_scrape.py	Telethon	Connects to Telegram channels (e.g., openai, ai_daily) and fetches recent messages asynchronously.
image_generator.py	Pollinations AI (Flux model)	Generates an image from a text prompt, saves it as a PNG file, and returns the public URL path.

9. Background Job Processing

Scheduled posts use BullMQ with Redis as the message broker:

- **Queue:** 'post-generation' — a single queue handles all scheduled post generation jobs.
- **Worker:** Started when the Express server boots (startWorker() in server.js). Processes one job at a time.
- **Job Lifecycle:**
 - 1. User creates a schedule → controller adds a delayed job to BullMQ with the appropriate delay until the first run.
 - 2. When the delay expires, the worker picks up the job and executes the full pipeline: scrape YouTube → send to LLM webhook → optionally generate image → save Post and ResearchReport to DB.
 - 3. If auto-publish is enabled, the worker calls publishToLinkedInInternal() to post directly to LinkedIn.
 - 4. The worker updates ScheduledTask.nextExecution and re-queues itself with a new delay for the next cycle.
 - 5. An email notification is sent to the user (success or failure) via the Brevo API.
- **Failure Handling:** If a job fails, the scheduled task is deactivated (isActive = false) and an error notification email is sent to the user.

10. Security Considerations

Measure	Implementation
Password Hashing	bcrypt with 10 salt rounds
Authentication	JWT tokens with 7-day expiry, verified on every protected route via middleware
Email Verification	6-digit OTP with 15-minute expiry and max 5 attempts

Measure	Implementation
Rate Limiting	express-rate-limit on OTP resend endpoint (3 per hour per IP)
OAuth2	Standard authorization code flow for LinkedIn; Google ID token verification on server
Input Validation	All endpoints validate required fields; banned words filter on AI generation prompts
Token Refresh	LinkedIn access tokens are auto-refreshed when within 60 seconds of expiry
CORS	Enabled via the cors middleware (should be restricted in production)
Secrets Management	All API keys and secrets stored in .env files, not committed to version control
Soft Delete	Posts are soft-deleted (deleted: true) to preserve data integrity

11. Application Flows

11.1 Signup & Email Verification Flow

- 1. User fills in email + password and submits the signup form.
- 2. Backend hashes the password (bcrypt) and generates a 6-digit OTP code.
- 3. OTP is stored temporarily in the EmailVerification table (expires in 15 minutes).
- 4. A verification email is sent via the Brevo SMTP API.
- 5. User is redirected to the email verification page.
- 6. User enters the OTP code; backend validates it, creates the User record, and deletes the temporary EmailVerification entry.
- 7. A JWT token is issued and the user is logged in automatically.
- Resend: Users can request a new OTP at any time. A fresh code is generated, the attempt counter is reset, and a new email is sent.

11.2 AI Post Generation Flow

- 1. User enters a topic/prompt and selects a tone (Neutral, Professional, Casual, or Viral).
- 2. The backend spawns the YouTube scraper Python script, which searches for relevant videos and collects video metadata + comments.
- 3. The scraped JSON data is POSTed to the n8n webhook URL, which uses an LLM to generate a post.
- 4. Optionally, the image_generator.py script is run to create an AI-generated image via Pollinations AI.
- 5. The generated post and research data are saved to the database (Post + ResearchReport).
- 6. The user can review the post, copy it, publish it to LinkedIn, or generate another.

11.3 Scheduled Posting Flow

- 1. User sets a topic, interval, start time, and optionally enables LinkedIn auto-publish.
- 2. A ScheduledTask record is created and a delayed BullMQ job is enqueued.
- 3. The worker processes the job at the scheduled time (scrape → generate → save).
- 4. If LinkedIn auto-publish is enabled, the post is published immediately.
- 5. The worker updates nextExecution and re-queues for the next interval.
- 6. An email notification is sent to the user.

12. Project Structure

SmartPost/

- └─ smartpost-app/ # React frontend (Vite)
 - | └─ src/
 - | | └─ App.jsx # Main app with routing and theme
 - | | └─ components/ # Feature components (8 modules)
 - | | | └─ Login/ # Login page
 - | | | └─ Signup/ # Registration page
 - | | | └─ EmailVerify/ # OTP verification page
 - | | | └─ Dashboard/ # Landing dashboard
 - | | | └─ Generate/ # AI post generation
 - | | | └─ Scheduler/ # Recurring schedule manager
 - | | | └─ LinkedIn/ # LinkedIn account & post management
 - | | | └─ Navbar/ # Navigation bar
 - | | └─ index.css # Global styles & theme
 - | └─ vite.config.js
 - └─ backend/ # Express.js API server
 - | └─ src/
 - | | └─ server.js # Entry point — starts Express + BullMQ worker
 - | | └─ app.js # Express app configuration & route mounting
 - | | └─ controllers/ # Business logic (4 controllers)
 - | | | └─ auth.controller.js
 - | | | └─ generate.controller.js
 - | | | └─ linkedin.controller.js
 - | | | └─ scheduler.controller.js
 - | | └─ routes/ # RESTful route definitions (6 modules)
 - | | └─ middleware/ # JWT authentication middleware
 - | | └─ queues/ # BullMQ queue + worker (postQueue.js)
 - | | └─ prisma/ # Prisma schema, client, and migrations

- | |— python/ # Python scraping & AI scripts (5 scripts)
- | |— .env # Environment configuration
- |— docs/ # Documentation & proposals

13. Installation & Deployment

13.1 Prerequisites

- Node.js (v18 or later)
- Python 3.10+ with pip
- MySQL 8.0
- Redis server (for BullMQ)
- npm (comes with Node.js)

13.2 Installation Steps

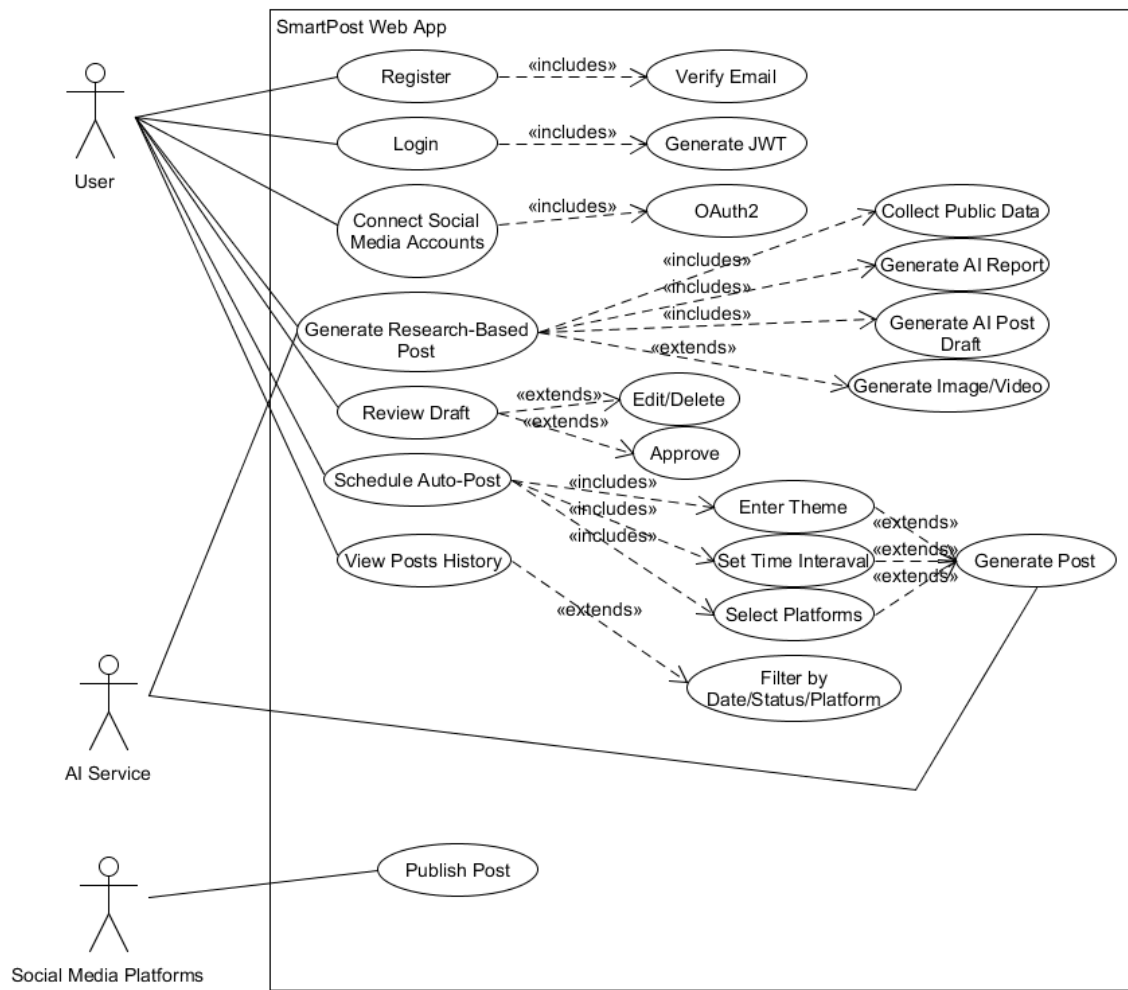
- **1. Clone the repository:** git clone <repo-url>
- **2. Frontend setup:** cd smartpost-app && npm install
- **3. Backend setup:** cd backend && npm install
- **4. Configure .env:** Set DATABASE_URL, JWT_SECRET, BREVO_API_KEY, LinkedIn OAuth credentials, YouTube API key, Redis connection, and other required variables.
- **5. Create database:** CREATE DATABASE smartpost; in MySQL
- **6. Run migrations:** npx prisma migrate dev
- **7. Install Python dependencies:** pip install google-api-python-client python-dotenv playwright telethon requests

13.3 Running the Application

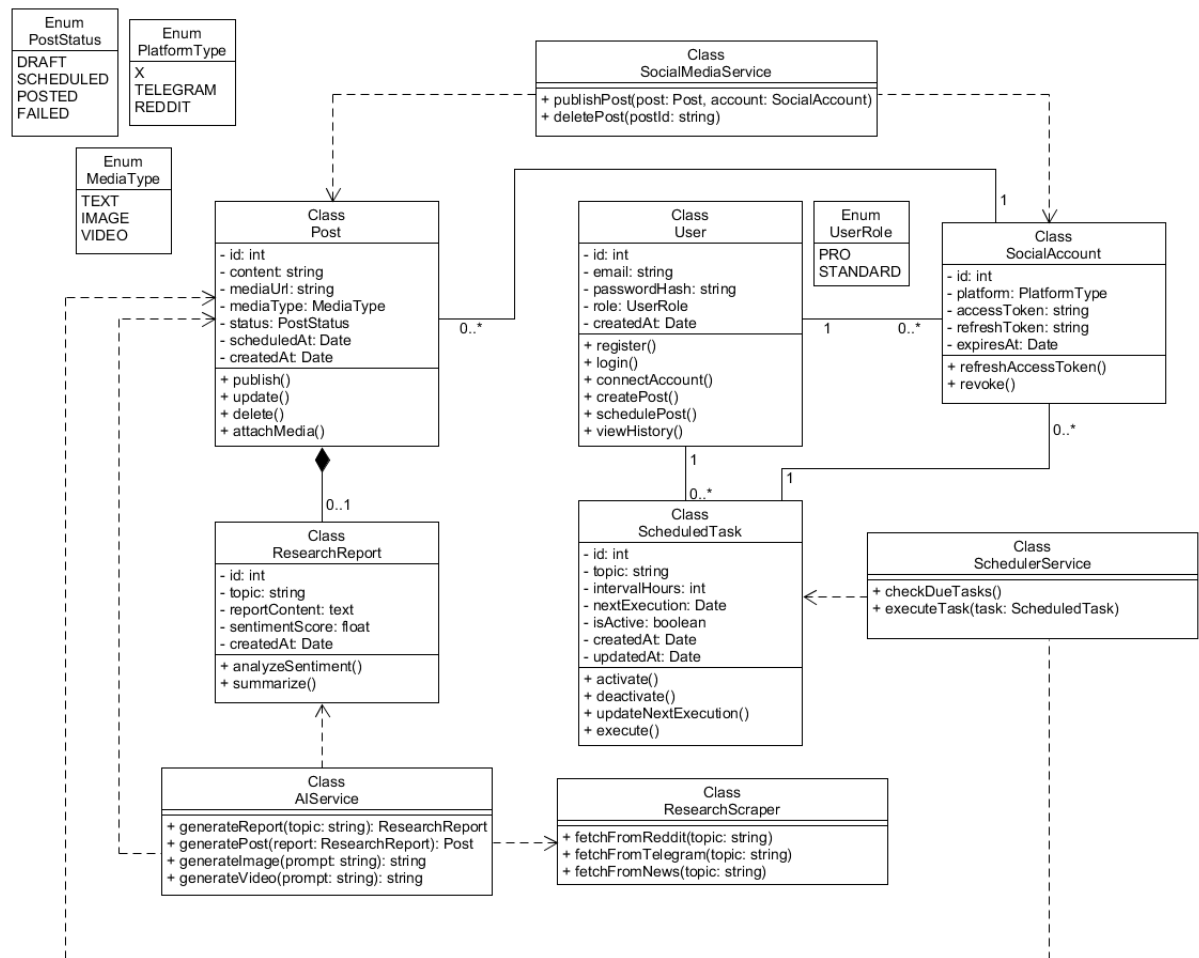
- Backend: cd backend && npm run dev (starts Express + BullMQ worker on port 4000)
- Frontend: cd smartpost-app && npm run dev (starts Vite dev server on port 5173)

14. Diagrams

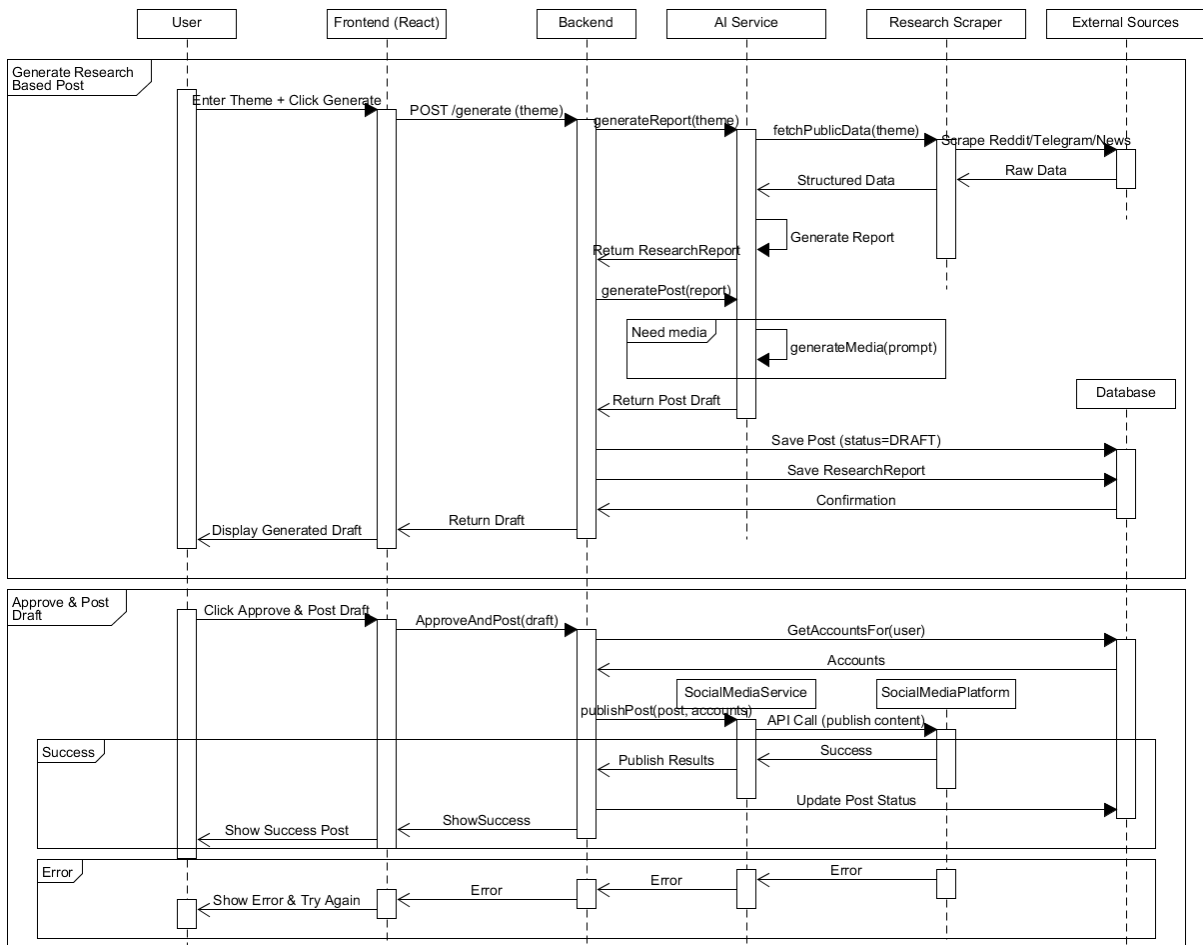
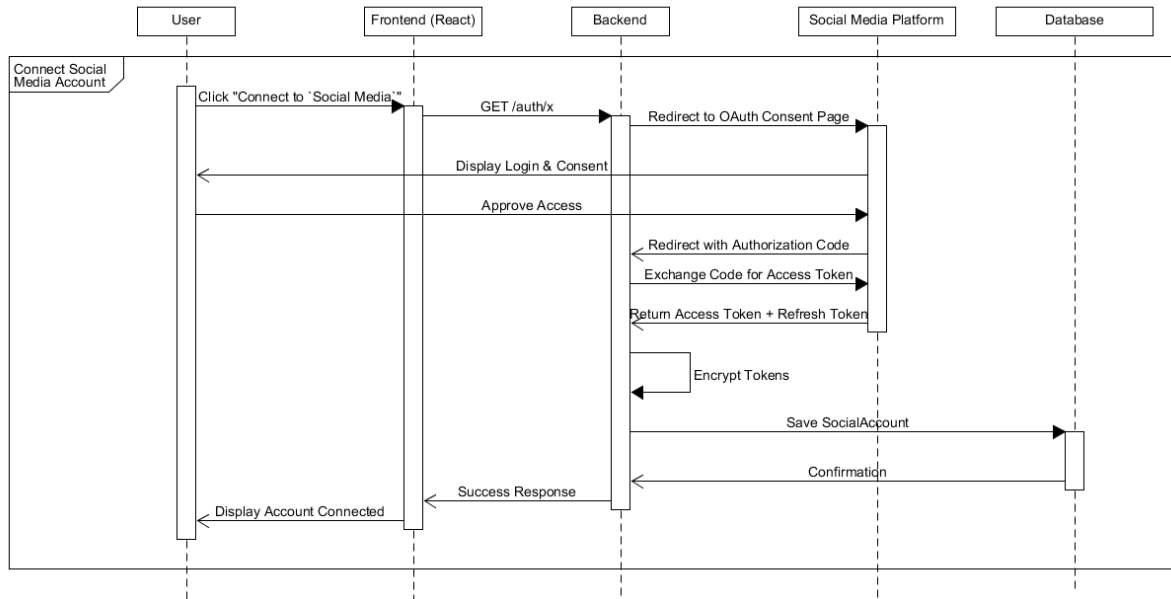
14.1 Use Case Diagram

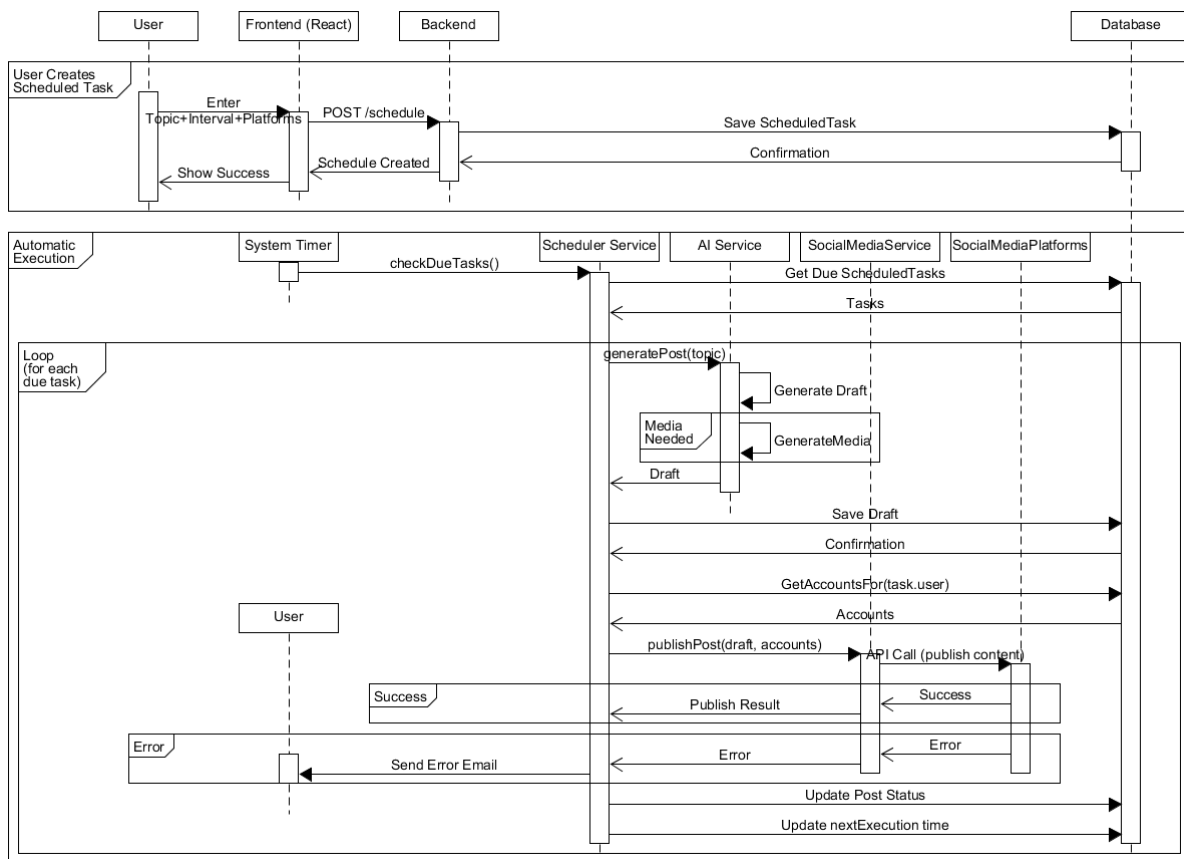


14.2 Class Diagram

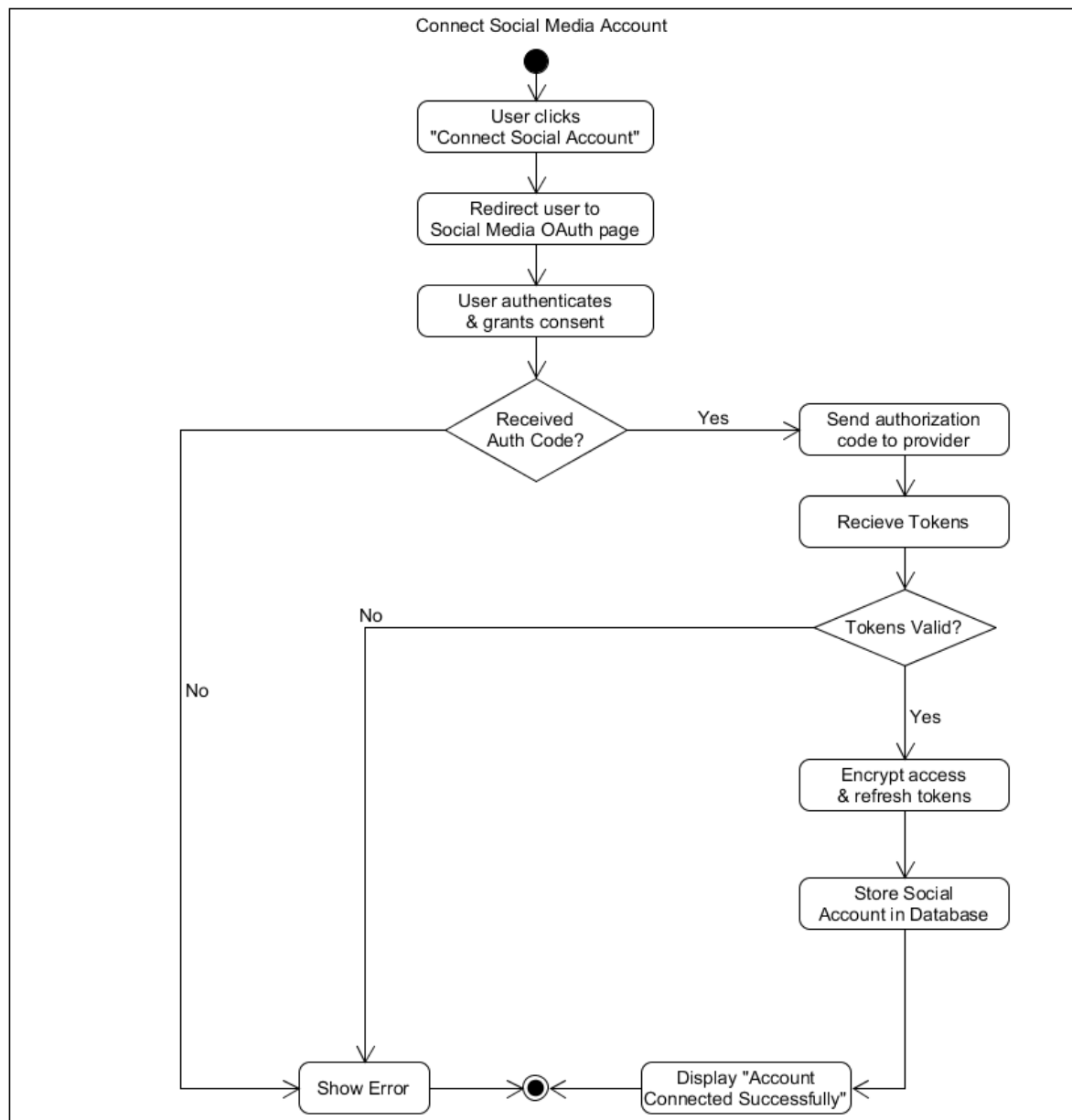


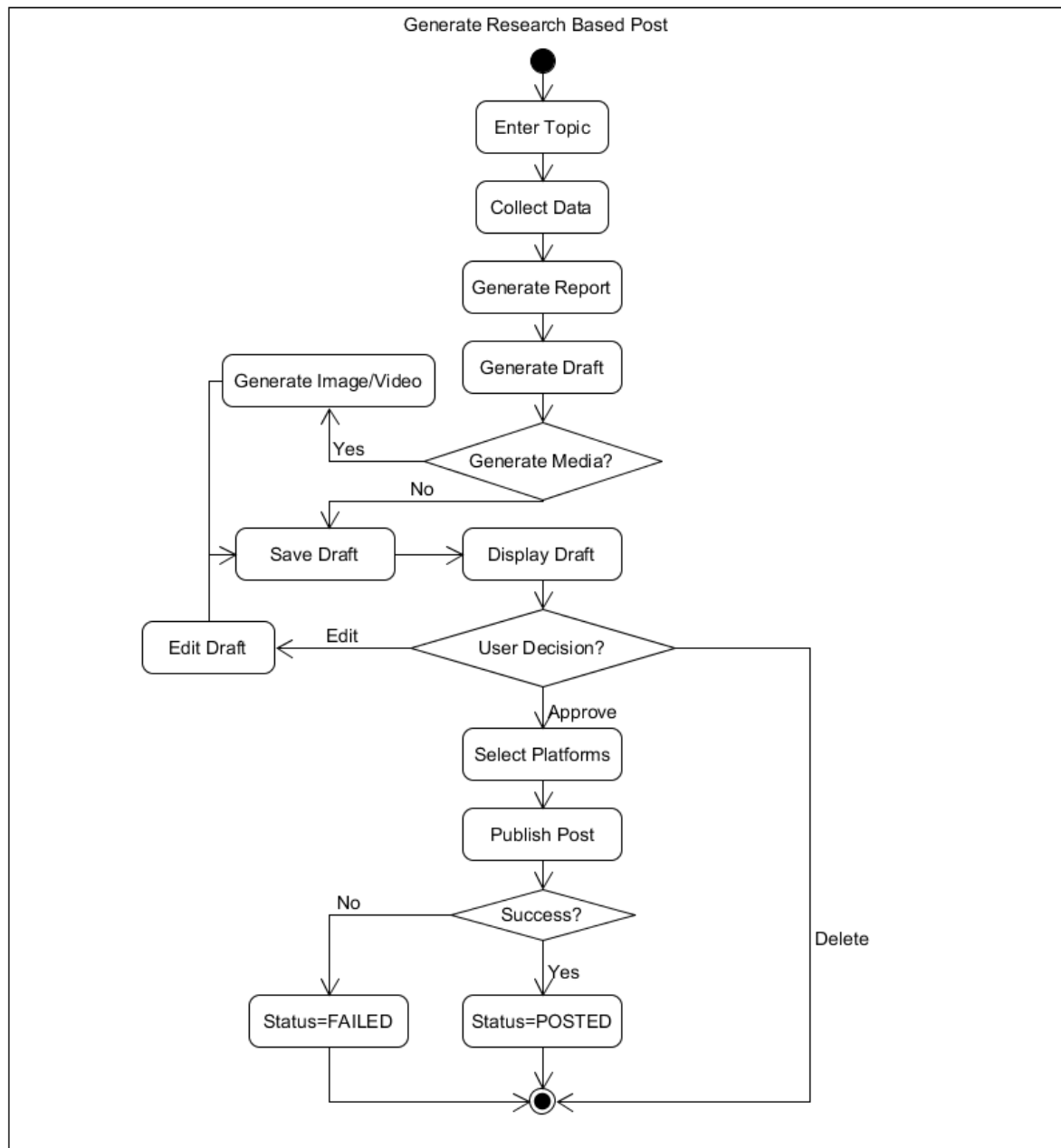
14.3 Sequence Diagram

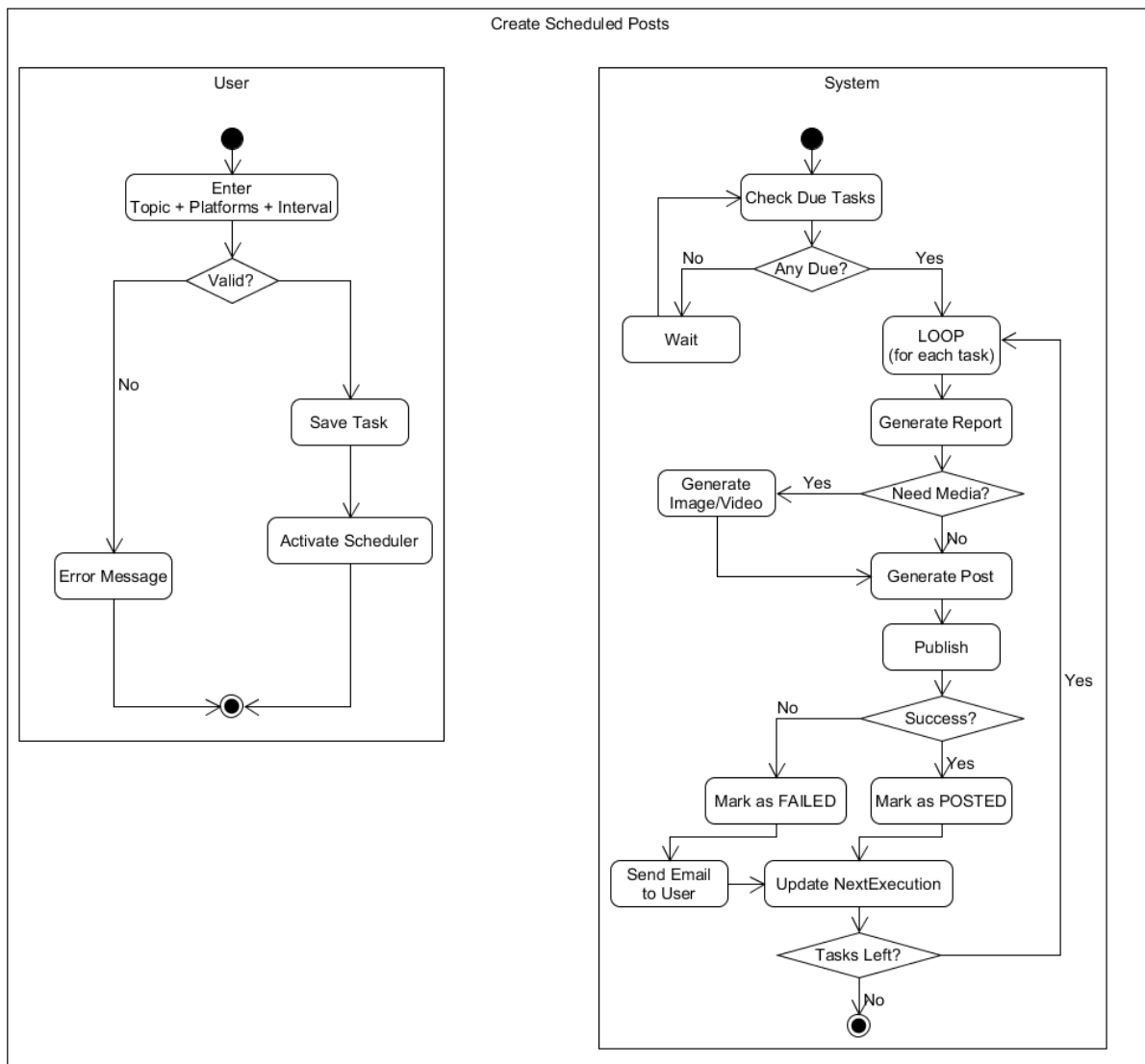




14.4 Flowchart







15. Appendix

A.1 System Architecture

The system architecture of SmartPost AI is designed around a decoupled, three-tier software paradigm consisting of a client-side presentation layer, a backend Application Programming Interface (API) server layer, and a service layer that communicates with external data providers, queues, and database engines. This modular separation of concerns ensures that the platform remains highly scalable, testable, and maintainable. It facilitates parallel

development of the React-based User Interface (UI) and the Node.js/Express API, and allows for independent optimization of the background worker tasks and web scrapers. This architectural approach avoids tight coupling between components, enabling system updates without breaking downstream integrations.

Furthermore, by organizing the backend into modular routes and controllers, the system can scale vertically and horizontally. For instance, in an enterprise scenario, the scraping components could be migrated to a dedicated set of microservices, while the central Express API acts as the secure request router and business logic coordinator. The frontend client operates as a static single-page application, minimizing server-side load and prioritizing browser execution speed and visual transitions.

A.1.1 Overall Architecture Explanation

SmartPost AI adheres to the Model-View-Controller (MVC) architectural design pattern, adapted for single-page applications. The React frontend serves as the View, managing the UI state, rendering pages dynamically, and catching user interaction events. The Express.js backend acts as the Controller, validating incoming API requests, managing session context, and directing the flow of business data. The database, managed through Prisma Object-Relational Mapping (ORM), forms the Model layer, abstracting the relational MySQL tables into type-safe JavaScript entities. This architecture minimizes data retrieval latency by utilizing Prisma's optimized query engine.

To process resource-intensive operations such as social media scraping and image generation without blocking the main event loop of the Express server, the application implements a Service-Oriented architecture. The background queue system uses BullMQ, a Redis-backed job queue manager, to coordinate scheduled auto-posting tasks. Spawning separate Python interpreter instances for YouTube, Facebook, Google, and Telegram scraping keeps browser automation and network-bound scraper tasks isolated, ensuring the Express server remains highly responsive to user requests. This isolation prevents memory leaks and CPU spikes in browser automation libraries from crashing the web server.

The integration of these services is coordinated through well-defined execution loops. For example, when a scraper is invoked, it is treated as a short-lived execution task that reports directly back to the controller via stdout. This allows the backend to implement timeouts, resource constraints, and sandboxing on the scraper scripts, preventing any single scraper from consuming excessive system resources.

A.1.2 Component Interactions

The system components interact through standard HTTP RESTful exchanges and background job invocations. When a user requests an AI-generated post through the React client, the following chronological steps occur:

- The React UI captures the user prompt, selected attitude (Neutral, Professional, Casual, Viral), and image toggle preference, sending them as a JSON payload to the `/api/generate` endpoint via Axios.
- The Express backend receives the request, passes it through the JWT authentication middleware to verify the user identity, and then validates the payload using a custom validator that checks for missing fields and compares the prompt against a banned words filter.

- Once verified, the generate controller spawns a Python child process running the `youtube_scraper.py` script, passing the prompt and attitude as CLI arguments.
- The Python script queries the YouTube Data API v3 and parses trending discussions, returning structured video descriptions, metadata, and comments as a stringified JSON string printed to stdout.
- The controller reads stdout from the child process, parses the JSON payload, and posts this data to an n8n orchestration webhook via a HTTP request.
- The n8n workflow parses the data, calls the external OpenAI LLM service to compile an analytical report and generate a social media post, and returns the output to the Express controller.
- If the user toggled the image generation option, the controller spawns a second child process running `image_generator.py`, which makes an HTTP request to the Pollinations AI service, downloads the generated PNG image, saves it to the backend public uploads folder, and returns the relative image URL path.
- The backend saves the newly created post as a 'draft' in the MySQL Post table and registers the raw JSON research metadata in the ResearchReport table, both linked via relational keys, before returning the generated post content and image URL to the client.
- The React client receives the JSON response and updates the UI state, displaying the generated post and image to the user for final manual review, copy, edit, or direct publication.

This architectural flow decouples scraping, processing, generation, and storage, ensuring that slow browser automation and network latency from external APIs do not degrade the client experience. The system is designed to degrade gracefully: if a scraper fails, the pipeline generates content using the available APIs or user prompt inputs.

A.1.3 Data Flow Descriptions

Data flow in SmartPost AI is categorized into two main pipelines: the synchronous client-server generation pipeline and the asynchronous background scheduler pipeline. In the client-server pipeline, data flows from the React interface as user input (prompts, preferences) to the Node.js API. The API processes this data by delegating search and extraction tasks to local Python scripts. These scripts return structured web scraping data back to Node.js, which then sends the structured data to n8n. The n8n automation engine orchestrates the call to the Large Language Model, and the generated content flows back through Node.js, gets written to MySQL, and is returned to the user's browser.

In the background scheduler pipeline, data flow is driven by the BullMQ worker. At the scheduled interval, the worker initiates the post-generation sequence using the stored ScheduledTask parameters. The generated post is stored in the database, and if the auto-

publish toggle is active, the data is pushed to the LinkedIn API. Finally, an execution status report is compiled and emailed to the user via Brevo's SMTP API. This decoupled data flow ensures that scheduling runs autonomously in the background without user intervention.

During this data flow, logging utilities record execution metadata at each boundary. If a network interruption occurs between the worker and the n8n webhook, the worker retries the request using backoff configurations before marking the job as failed and notifying the user. This ensures the data flow remains reliable under varying network conditions.

A.1.4 Client-Server Communication

Client-server communication is achieved exclusively using JSON-formatted HTTP request and response payloads. The client communicates with the server via standard HTTP verbs: GET for fetching posts, drafts, and schedule details; POST for submitting authentication credentials, generating posts, and establishing schedules; PATCH for updating existing posts; and DELETE for deauthorizing platforms or cancelling schedules. All requests to protected endpoints must contain the JWT bearer token in the Authorization header. Cross-Origin Resource Sharing (CORS) is enabled on the server to permit requests from the Vite frontend port, with proper restrictions configured for production deployment.

To optimize bandwidth and response times, responses are structured to exclude unnecessary details. For example, large JSON research logs are stored in the database but are not returned in standard list operations, being retrieved only when the user requests a detailed report view. This minimizes network overhead and improves page load times.

[Placeholder: Figure A.3: Authentication Flow]

Description: A detailed sequence diagram illustrating the client, server, and LinkedIn OAuth server communication flow during authentication and API authorization exchange.

A.2 Database Design

The data layer of SmartPost AI is built using MySQL 8.0, a relational database management system. Access to the database is managed via Prisma ORM, which provides type safety, automatic schema migrations, and relational abstractions. The database design emphasizes data integrity, relational consistency, and quick query response times through indexes and cascade delete rules.

By defining database schemas in a declarative schema.prisma file, developers can version-control the database structure. Schema modifications are translated into SQL migration scripts that are executed during deployment, ensuring that all environments remain synchronized.

A.2.1 Entity Descriptions

The database schema consists of seven main tables, each representing a core domain entity of the application. The primary entities are:

- **User:** Stores the credential and role data of registered accounts. Passwords are stored in hashed formats using bcrypt. The role field defines access privileges (standard or pro).
- **Platform:** Represents the social media platforms supported by the system (e.g., LinkedIn, Smartpost Default), holding base API URLs and activation flags.
- **SocialAccount:** Resolves the many-to-many relationship between Users and Platforms. It securely stores access and refresh tokens, expiration timestamps, and member profile details like URNs and names.
- **Post:** Represents the generated posts, whether drafts, scheduled, posted, or failed. It contains the content commentary text, associated media URLs, post types, external publication IDs, and soft-delete flags.
- **ResearchReport:** Stores the raw JSON scrapings obtained from the Python web scrapers. This table provides historical research material and is linked to its corresponding generated Post.
- **ScheduledTask:** Holds information regarding recurring post-generation routines, including the prompt topic, interval frequency, next execution date, and auto-publishing parameters.
- **EmailVerification:** Holds temporary email verification entries, security codes, and attempt counters. Rows are purged upon successful user activation.
- **Draft:** Stores independent, manually saved content drafts that are not directly tied to scraping reports.

A.2.2 Relationships Between Tables

The relationships between the tables are configured to enforce strict relational boundaries. A User can have one or many SocialAccounts (one-to-many), allowing multiple platform accounts to be linked to a single user. A Platform can have many linked SocialAccounts (one-to-many). The combination of `userId` and `platformId` in the SocialAccount table has a unique index, preventing a user from connecting duplicate accounts for a single platform. Each Post is associated with one SocialAccount (many-to-one), tracking which platform and profile published it. A Post has a one-to-many relationship with ResearchReport, keeping historical scraping reports linked to the generated post. A SocialAccount is also linked to ScheduledTasks (one-to-many), binding recurring automation tasks to a specific social profile. Cascade deletes are set on all foreign keys: if a user deletes their profile or disconnects an account, all linked posts, schedules, and research reports are purged automatically to preserve storage space.

This relational structure is defined using Prisma's relation fields. For instance, the cascade deletion rules are declared in the Prisma schema, which Prisma compiles into SQL FOREIGN KEY constraints with ON DELETE CASCADE. This shifts the responsibility of data cleanup to the database engine, minimizing backend code complexity and preventing orphaned rows.

A.2.3 Database Schema Explanations

To optimize data access, performance indexes are configured on frequently queried columns. The Post table contains indexes on the status and scheduledAt fields, allowing the backend to quickly query drafts, scheduled posts, and historical listings. The ScheduledTask table features indexes on the isActive and nextExecution fields, allowing the background worker to scan active tasks scheduled for execution. The Draft table contains an index on the userId column. Data types are configured for precision: primary keys use unsigned BigInt fields for maximum ID capacity, text columns like post content use TEXT, access tokens use TEXT, and research reports use LONGTEXT to handle large JSON objects.

The use of BigInt fields for identifiers requires specific serialization handling. Because JavaScript cannot natively serialize BigInt values to JSON without throwing errors, the backend implements serialization logic that converts all BigInt IDs to standard strings before sending responses to the client. This ensures compatibility with frontend clients.

A.2.4 Sample Records and Field Descriptions

The structure and properties of the database entities are detailed in the schema reference table below.

Entity Table	Field Name	Data Type	Constraints / Key	Description
User	id	BigInt Unsigned	Primary Key, AutoIncrement	Unique user identifier
User	email	VarChar(255)	Unique Index	User login email address
User	password	VarChar(255)	Required	Bcrypt hashed password
User	role	Enum('pro','standard')	Default: 'standard'	Access role for permissions
Platform	id	BigInt Unsigned	Primary Key, AutoIncrement	Unique platform identifier

Platform	platformName	VarChar(100)	Unique Index	Name of social platform
Platform	apiBaseUrl	VarChar(255)	Required	Base URL for platform API requests
SocialAccount	id	BigInt Unsigned	Primary Key, AutoIncrement	Unique account mapping ID
SocialAccount	userId	BigInt Unsigned	Foreign Key -> User.id	Owner of the social account
SocialAccount	platformId	BigInt Unsigned	Foreign Key -> Platform.id	Linked platform ID
SocialAccount	accessToken	TEXT	Required	OAuth access token
SocialAccount	refreshToken	TEXT	Optional	OAuth refresh token
SocialAccount	linkedinUrn	VarChar(100)	Optional	LinkedIn unique profile URN
Post	id	BigInt Unsigned	Primary Key, AutoIncrement	Unique post identifier
Post	socialAccountId	BigInt Unsigned	Foreign Key -> SocialAccount.id	Linked social account
Post	content	TEXT	Optional	Text body of the post
Post	status	Enum('draft','posted','failed')	Default: 'draft'	Publication lifecycle status
ScheduledTask	id	BigInt Unsigned	Primary Key, AutoIncrement	Unique task identifier
ScheduledTask	topic	VarChar(255)	Required	Search keyword topic for AI generation
ScheduledTask	intervalHours	Int Unsigned	Required	Task execution frequency in hours
ScheduledTask	nextExecution	DateTime	Indexed	Next execution timestamp

[Placeholder: Figure A.4: Database Schema]

Description: Entity-Relationship diagram illustrating the database tables, fields, constraints, indexes, and primary/foreign key mappings within MySQL.

A.3 API Documentation

The SmartPost AI backend exposes a RESTful API to enable client communication and external system integration. The API uses JSON for all request payloads and responses. The endpoints are organized by resource domains: authentication, generation, scheduler, LinkedIn integration, and system health status.

Every API route applies strict payload validation middleware. If the request body does not match the expected structure, the server terminates the request early and returns an HTTP 400 Bad Request status code. This checks parameters before executing database queries or spawning external processes.

A.3.1 Authentication Mechanisms

All protected API endpoints require JWT authentication. When a user logs in or verifies their email successfully, the backend issues a signed JWT token containing the user's ID, email, and role. The token is signed using the server's private secret and expires after 7 days. The client must store this token and attach it to the header of all subsequent API calls. The format must conform to the standard HTTP schema: Authorization: Bearer <token>. For redirect-based browser callback flows where setting headers is not possible (such as LinkedIn login), the token is temporarily passed as a query string parameter and verified by the auth middleware.

Security configurations include rate limits for authentication requests and verification codes. Additionally, the JWT validation middleware catches signature mismatch errors, token expiration, and malformed strings. Upon catching these, it returns an HTTP 401 Unauthorized response, protecting internal endpoints from unauthenticated access.

A.3.2 Endpoint Descriptions

The following reference table documents the paths, HTTP methods, access rights, and functional descriptions of the API endpoints.

Method	Endpoint	Auth	Request Payload Details	Response Details
POST	/api/auth/signup	No	email, password	Confirmation message OTP sent
POST	/api/auth/verify-email	No	email, code	JWT Token and user profile JSON
POST	/api/auth/resend-code	No	email (Rate-limited: 3/hr)	Confirmation message of OTP resend
POST	/api/auth/login	No	email, password	JWT Token and user profile details
POST	/api/auth/google	No	token (Google ID Token)	JWT Token or verification requirement
POST	/api/generate	Yes	prompt, attitude, includeImage	Generated post text, image URL, counts
GET	/api/linkedin/login	Yes	token passed as query	Redirect to LinkedIn consent page

GET	/api/linkedin/callback	No	code, state, error (Query params)	Redirect to frontend URL status
GET	/api/linkedin/status	Yes	None	JSON details of connected LinkedIn profile
GET	/api/linkedin/posts	Yes	None	JSON list of non- deleted posts
POST	/api/linkedin/post	Yes	text, optional image file	Success message, created post ID
PATCH	/api/linkedin/post/:id	Yes	text	Success status of updated commentary
DELETE	/api/linkedin/post/:id	Yes	None	Success status of soft-deleted post
DELETE	/api/linkedin/disconnect	Yes	None	Account disconnected confirmation
POST	/api/scheduler	Yes	topic, attitude, intervalHours, scheduledAt	Created task JSON object
GET	/api/scheduler	Yes	None	JSON list of active user schedules
DELETE	/api/scheduler/:id	Yes	None	Cancellation confirmation status
GET	/api/health	No	None	Server health and uptime status

A.3.3 Request and Response Examples

This section provides examples of request payloads and successful JSON responses for core API endpoints.

Example: POST /api/generate (Request)

```
{
  "prompt": "Artificial Intelligence in Healthcare",
  "attitude": "Viral",
  "includeImage": true
}
```

Example: POST /api/generate (Response)

```
{
  "post": "🤖 AI is revolutionizing healthcare! From predictive analytics to robotic surgery, the future of medicine is here. #AI #HealthcareTech",
  "imageUrl": "/generated_images/ai_healthcare_17186400.png",
  "imageFilename": "ai_healthcare_17186400.png",
  "scrapedCount": 5,
  "sources": {
    "youtube": 5
  }
}
```

```
}  
}
```

Example: GET /api/linkedin/status (Response)

```
{  
  "connected": true,  
  "name": "Toufik Dawaliby",  
  "urn": "urn:li:person:89XyZ7w9"  
}
```

A.3.4 Error Handling and Status Codes

The API uses standard HTTP response status codes to communicate the success or failure of operations. Client errors return a 400 Bad Request if parameters are missing or invalid (such as invalid scheduled times or OTP codes). Unauthorized access attempts return a 401 Unauthorized, and access to forbidden resources returns a 403 Forbidden. A 404 Not Found is returned for missing posts or schedules, and a 429 Too Many Requests is returned if rate limits are exceeded (for example, on the /api/auth/resend-code endpoint). Internal server errors return 500 Internal Server Error, along with a generic message and error details logged on the server. All error responses follow a standard format: { "message": "Detailed error description" }.

Furthermore, error handlers intercept unhandled promise rejections on the Express app instance. This ensures that even unexpected system anomalies do not cause the process to crash, maintaining system uptime.

A.4 User Interface Screens

The frontend application provides a single-page dashboard designed to simplify social media content creation and automation. The user interface uses a responsive layout, custom typography, consistent spacing, and automatic dark/light theme options.

The design system utilizes HSL-tailored color schemes instead of generic browser primary colors. This establishes a clean interface that adapts to the system theme mode. Modern transitions and subtle micro-animations enhance user interaction, providing clear feedback on button hovers, page transitions, and loading states.

A.4.1 Page Descriptions and Features

The interface consists of several core pages and modal panels:

- **Login / Signup:** Contains input forms for credentials, styled validation prompts, and an alternative Google sign-in button.
- **Email Verification:** Displays a 6-digit OTP entry field, expiration countdown timer, error messages, and a rate-limited resend button.
- **Dashboard (Home):** Presents a card-based grid layout offering entry points to key modules. It displays usage statistics, system status, and user recommendations.
- **AI Post Generator:** Provides a prompt input form, an attitude dropdown selector, an image generation toggle switch, and a dynamic loading spinner. The generation results are shown in a card with options to copy, edit, or publish directly.
- **Post Scheduler:** Displays a scheduler form to configure prompts, execution intervals, start times, and auto-publishing options. An active tasks panel lists running schedules with cancellation buttons.
- **LinkedIn Manager:** Provides connection controls (connect/disconnect account), profile summaries (avatar placeholders and names), and a history feed showing generated posts, publication statuses (draft, posted, failed), and action buttons.

A.4.2 Buttons, Controls, and Input Forms

Interactive UI controls are designed to prevent invalid user inputs. Form submission buttons show loading states and disable automatically during active operations to prevent duplicate submissions. Dropdowns (such as the attitude selector) use predefined options, and the date-time scheduler input restricts selections to future dates. Action buttons, like 'Delete Post' or 'Disconnect Account,' trigger confirmation dialogs to prevent accidental data loss. Toast notifications provide immediate feedback on success or failure, and copy-to-clipboard buttons provide instant confirmation.

Each input form is configured with keyboard listeners. For example, pressing the enter key in the generator prompt input triggers the generation process automatically. Focus rings and keyboard navigation options ensure the interface remains accessible to all users.

A.4.3 User Workflows and Navigation

The standard user workflow begins with registration. The user submits their email and password, which redirects them to the Email Verification page. Upon entering the 6-digit OTP received via email, the user is redirected to the Dashboard. From there, they can navigate using the sidebar or navbar. To publish content, the user first links their LinkedIn account via the LinkedIn page. Once connected, they can use the AI Post Generator to create content, edit the output in the preview card, and post it to their feed. For automation, they configure a recurring schedule on the Scheduler page, which handles generation and publishing automatically.

Navigation is managed using React Router's dynamic routing engine, enabling instant page transitions without browser page reloads. Private routes block unauthenticated access, redirecting unauthorized traffic to the login screen and preserving user state across pages.

[Placeholder: Figure A.1: Homepage Interface]

Description: Interface mockup showing the primary dashboard layout, navigation menus, quick statistics panels, and links to the content tools.

[Placeholder: Figure A.2: LinkedIn Post Creation Page]

Description: Screenshot placeholder representing the content creator screen, highlighting prompt configuration forms, tone selection dropdowns, and publication triggers.

A.5 Implementation Details

SmartPost AI is built using modern JavaScript libraries and runtime frameworks. The front-end uses a React-based structure, while the backend uses a Node.js runtime environment with helper scripts in Python.

A.5.1 Frontend Technologies and Libraries

The user interface is built using React 19.2 and Vite 7.3. Vite serves as the build tool, offering fast Hot Module Replacement (HMR) and optimized production bundles. React Router DOM 7.13 is used for routing, managing navigation between dashboard features and authentication guards. Axios 1.13 is used for HTTP communications, configured with interceptors to attach JWT authentication headers. Icons are provided by Lucide React 1.11, and Google OAuth integration is managed using @react-oauth/google 0.13. The styling system is built using custom Vanilla CSS with CSS variables, enabling light and dark theme modes. The theme is applied via a global HTML data attribute and managed using a React Context Provider.

The frontend avoids external styling frameworks like TailwindCSS to maintain complete control over layout styles. By utilizing CSS grid and flexbox, layouts adapt dynamically to varying screen sizes, ensuring compatibility with mobile devices, tablets, and large screens.

A.5.2 Backend Architecture and Packages

The backend API runs on Node.js v18+ with Express 5.2. Node.js's non-blocking, event-driven I/O model makes it well-suited for coordinating API endpoints, database operations, and background jobs. Express routes are organized by domain, and requests are passed to controllers that handle the business logic. Security packages include bcrypt 6.0 for password hashing and jsonwebtoken 9.0 for signing and verifying tokens. Database interactions are managed using Prisma 6.19 and its Client. For scheduled tasks, the backend uses BullMQ 5.71 and ioredis 5.10 to process tasks asynchronously, preventing long-running scrape jobs from blocking the main server loop. Email notifications are managed via Brevo's REST SMTP API, and sensitive endpoints are protected using express-rate-limit 8.2.

The server configuration mounts routes under a centralized API route prefix (/api). Middleware functions handle requests globally, checking CORS rules, parsing JSON payloads, logging traffic, and parsing multipart file uploads.

A.5.3 State Management and Hooks

The client application uses React's built-in hooks for state management instead of external libraries like Redux, keeping the architecture lightweight. Component states, such as input values, validation flags, and loading indicators, are managed using useState. Side-effects, including fetching post listings or checking LinkedIn connection status on mount, are handled by useEffect. The application's theme state (light/dark mode) and user authentication credentials (JWT tokens, user profiles) are made globally accessible across components using useContext, which pulls values from dynamic Theme and Auth Context providers.

To optimize form submissions and input responsiveness, useRef is utilized to track form inputs without triggering component rerenders on every keystroke. This improves input latency and overall page performance.

A.5.4 External Integrations and APIs

The platform integrates with five external service providers:

- YouTube API: Used by the scraper script to retrieve video metadata, metrics, and top comments.
- n8n Webhook: Serves as the AI orchestration engine, accepting scraping payloads and using OpenAI's LLM to generate posts.
- Pollinations AI: Used to generate images from text descriptions, returning a PNG file path.
- LinkedIn REST API: Used for OAuth2 login, profile data lookup, and publishing, editing, or deleting posts.
- Brevo API: Used to send transactional emails, including verification codes and task execution alerts.

The table below summarizes the key technical components of the platform.

Layer / Module	Technology Used	Role in System	Key Features / Dependencies
Frontend UI	React 19.2 + Vite 7.3	Single-Page Presentation Layer	HMR, component hierarchy, modular hooks
Client Routing	React Router DOM 7.13	Client-Side Route Management	Navigation guards, auth redirects, layout routing
Client Styling	Vanilla CSS	Theme and Layout Styling	CSS custom variables, flexbox/grid layout, transitions
Backend REST API	Node.js + Express 5.2	REST Server and Middleware Controller	Middleware pipeline, JSON routing, process spawn
ORM	Prisma 6.19	Database Abstraction and Migrations	MySQL mapping, migrations, database queries
Job Queue	BullMQ 5.71 + Redis	Background Worker and Queue Manager	Redis storage, retry limits, delayed jobs
Email SMTP	Brevo API / SMTP	Transactional Email Dispatcher	OTP notifications, execution failure notifications
Auth Hashing	Bcrypt 6.0	Hashed Password Persistence	10 salt rounds, safe registration validations
OAuth Services	Google Auth / LinkedIn API	Third-Party Authentication and Posting	OpenID Connect, Bearer access tokens, URN publishing
Web Scraping	Playwright + YouTube Data API	Data Scraping and Insights Gathering	Browser automation, JSON formatting, data extractors

AI Generators	Pollinations AI / n8n Webhook	AI-Powered Text and Image Creation	Prompt formatting, Flux generation, LLM orchestrator
---------------	-------------------------------	------------------------------------	--

A.6 LinkedIn Integration

LinkedIn is the primary social media platform integrated into SmartPost AI. The integration handles user authentication via OAuth 2.0, manages access tokens, and uses LinkedIn's REST API for publishing, editing, and deleting posts, including support for image uploads.

A.6.1 OAuth 2.0 Authentication Process

The authorization process follows the standard OAuth 2.0 authorization code flow. When a user connects their account, the application redirects them to LinkedIn's authorization server. The redirect URL includes the application's client ID, requested permissions (openid, profile, email, w_member_social), a redirect URI, and the user's ID as the state parameter to prevent CSRF attacks. After the user grants permission, LinkedIn redirects back to the /api/linkedin/callback endpoint with an authorization code. The backend exchanges this code for an access token via a POST request, retrieves the user's LinkedIn profile details, and saves the credentials to the SocialAccount table.

Retrieving member profiles utilizes the OpenID Connect /v2/userinfo endpoint. This retrieves the member's unique identifier (sub) and full name. The backend formats the identifier into a LinkedIn URN (urn:li:person:<sub ID>), which is required as the author identifier in all subsequent publishing requests.

A.6.2 Access Token Management and Lifetime

LinkedIn access tokens are valid for 60 days. To ensure uninterrupted automated posting, the backend implements automatic token refresh logic. The application checks the token's expiration date before each API call. If it is within 60 seconds of expiring, the server makes a POST request to LinkedIn's token endpoint using the refresh token. The new access token and updated expiration date are saved to the database. If a token refresh fails, the schedule is deactivated, and the user is prompted to reconnect their account.

The refresh token is retrieved from the initial OAuth code exchange when the offline_access scope is requested. Unlike the access token, the refresh token has a lifetime of 365 days. The automatic token refresh mechanism resets the 60-day access token window, keeping the account connected for up to a year without requiring manual user re-authentication.

A.6.3 Post Creation Workflow

To publish a post, the application makes a POST request to the LinkedIn Share API (/rest/posts) with the user's access token and API version headers (LinkedIn-Version: 202603). The request body specifies the author URN, the post content text, public visibility settings, and any associated media attachments. On success, LinkedIn returns a 201 Created status code along with the unique post URN in the x-restli-id header, which is saved locally to enable editing and deletion.

When editing commentary, the application makes a POST request to /rest/posts/<post URN> using a patch payload format. By attaching the header X-HTTP-Method-Override: PATCH, the server overrides the POST action, updating the post on LinkedIn's servers.

A.6.4 Image Upload and Binary Transmission

Publishing an image requires a three-step upload sequence:

- **Initialize Upload:** The backend makes a POST request to `/rest/images?action=initializeUpload`. The payload specifies the owner URN. LinkedIn returns an upload URL and a target image asset URN.
- **Upload Binary Data:** The backend reads the generated image from disk and sends the binary data to the provided upload URL via a PUT request with the appropriate MIME content-type header.
- **Create Post:** The backend sends a POST request to `/rest/posts`, attaching the image asset URN to the content block of the payload. This links the uploaded image to the new social media post.

This binary transmission uses streams to handle large image files. The server reads the file directly from the filesystem and streams the raw bytes directly to the upload URL, preventing high memory usage during concurrent uploads.

A.6.5 Error Handling and Security Considerations

The integration handles API failures, rate limits, and network errors gracefully. If LinkedIn returns an error code, the backend logs the response, marks the post status as 'failed', and sends an email notification to the user. Security measures include storing all OAuth credentials and client secrets in environment variables on the server. Access tokens are encrypted before being written to the database to protect them from unauthorized access.

To comply with LinkedIn's platform security policies, tokens are encrypted in the database using AES-256-GCM. The encryption key is stored securely in environment variables, protecting the tokens in the event of database access.

A.7 AI Content Generation Module

The content generation module uses artificial intelligence to turn raw research data into social media posts. The system gathers relevant information, structures it into JSON format, and calls external AI services to generate text and images based on the user's prompt.

A.7.1 Prompt Generation Process

The prompt generation process starts when a user submits a search topic and selects a tone. The backend validates the prompt against a banned words list. If approved, the prompt is passed to the YouTube scraper script, which returns video metadata and top comments. The system then formats this data into a structured payload for the n8n orchestration service. The payload includes the original prompt, the scraping results, and instructions to generate a post that matches the user's selected tone (Neutral, Professional, Casual, or Viral).

The prompt instructions sent to the LLM specify structure, character limits, hashtag limits, and call-to-action constraints. For example, a Viral prompt instructs the model to include hooks, bulleted summaries, and trending hashtags, while a Professional prompt focuses on analytical language and structured summaries.

A.7.2 Workflow and Processing Steps

The post-generation pipeline follows these steps:

- **Validation:** The prompt is checked against the banned words list.
- **Scraping:** A Python child process searches YouTube and extracts video descriptions and comments.
- **Orchestration:** The scraped data is posted to n8n, which formats it and calls the LLM.
- **Image Generation:** If selected, the backend spawns a process to generate an image using Pollinations AI.
- **Saving:** The generated post is saved to the database as a draft, and the scraped data is logged in the ResearchReport table.
- **Response:** The backend returns the post content and image path to the frontend.

A.7.3 Integration with External AI Services

The system uses two external AI integrations: n8n for text generation and Pollinations AI for image generation. n8n handles the LLM orchestrations, using OpenAI models to summarize the research data into a social media post. Pollinations AI uses the Flux image model to generate visual assets from a text prompt. The image is downloaded, saved to the server's public upload directory, and served statically.

By routing text generation through n8n, developers can modify prompt templates, switch AI models, or add processing steps in a visual canvas without changing the core backend code. This decoupled design makes it easy to integrate alternative models like Claude or Gemini.

A.7.4 Response Handling and JSON Parsing

The backend uses structured error handling to parse output from child processes. Python scripts output structured JSON strings to stdout. The backend captures this output, filters out log messages, and parses the JSON. If a script fails or prints invalid JSON, the system catches the error, logs the failure, and falls back to a template post to prevent application crashes.

The parsing system scans the output buffer for lines starting with opening brackets. This allows the backend to filter out debug messages or warnings printed by Python libraries, extracting only the clean JSON payload.

A.8 Redis and Caching Mechanisms

SmartPost AI uses Redis to manage background tasks and ensure reliable job execution. Redis serves as the message broker for BullMQ, which handles scheduled post-generation tasks.

A.8.1 Purpose of Redis in SmartPost

Redis is an in-memory data store used to manage the BullMQ task queues. It acts as a message broker, tracking active, delayed, completed, and failed tasks. This design keeps long-running background tasks (like scraping and image generation) out of the main Express server, ensuring the API remains responsive to client requests.

Using Redis for task management ensures that scheduled jobs are persisted. Even if the Node.js API server restarts, active and delayed job schedules remain saved in the Redis database, resuming automatically when the server boots.

A.8.2 Data Storage Strategy and Configurations

Redis organizes queue data using key-value hashes, lists, and sorted sets. BullMQ uses sorted sets to schedule delayed jobs based on their execution times. When a job is ready, it is moved to an active list for processing. Redis holds job data, configurations, and retry limits, protecting the queue status from backend restarts.

The application connects to Redis using connection options defined in environment variables. Connection settings configure timeouts, keep-alive parameters, and reconnection rules, preventing connection drops from blocking background tasks.

A.8.3 Queue Handling and Background Workers

A BullMQ worker starts when the Express server boots. The worker listens to the post-generation queue and processes jobs sequentially. When processing a job, the worker runs the scraper, generates the post, updates the ScheduledTask database record with the next execution time, and schedules the next job in the queue. If a job fails, the worker logs the error, deactivates the schedule, and emails a notification to the user.

The queue configuration specifies limits for completed and failed job histories. This prevents Redis memory usage from growing excessively over time by automatically cleaning up old records.

A.8.4 Session Management and System Reliability

The system handles server restarts and network disconnects gracefully. Active job states are persisted in Redis. If the backend restarts, the worker reconnects to Redis and resumes processing any pending or delayed tasks. This persistent design ensures that scheduled posts are executed even during server updates or temporary downtime.

To maintain reliability, the backend uses connection pools for database interactions. Database connections are recycled automatically, preventing connection leaks from degrading database performance.

A.9 n8n Automation Workflows

n8n is the workflow automation tool used to orchestrate the AI content generation pipeline. It processes the raw scraped data and calls the LLM to generate the final post text.

A.9.1 Workflow Descriptions

The n8n workflow starts with a webhook node that listens for incoming POST requests. Once triggered, the workflow passes the scraped research payload to an OpenAI node. This node instructs the LLM to analyze the comments and metadata, summarize the key discussions, and write a social media post in the requested tone. The workflow then serializes the generated text into a JSON payload and returns it in the HTTP response.

Within the n8n editor, nodes are connected to handle errors and format data. For example, if the OpenAI node fails, the workflow routes to a fallback node that returns a generic template post, preventing pipeline crashes.

A.9.2 Trigger Mechanisms

The n8n workflow is triggered by an HTTP POST request from the Express backend's generate controller or background worker. The request body contains the raw JSON scraping data (such as YouTube comments, views, and descriptions) and the user's prompt and tone preferences. The backend secures the endpoint by validating the request source and payload structure.

To protect the n8n endpoint from unauthorized access, the request header must contain an API key verified by n8n. Requests without valid keys are blocked immediately.

A.9.3 Data Processing Stages

The workflow processes data in three stages:

- **Ingestion:** The webhook node parses the incoming JSON request.
- **Analysis & Generation:** The OpenAI node processes the prompt, tone, and research data to write the post.
- **Response:** The workflow formats the generated text and returns it to the backend.

During the generation stage, a prompt template combines the user request with instructions to generate structured outputs containing commentary, hashtags, and formatting.

A.9.4 Integration with Third-Party Services

The n8n workflow connects to the OpenAI API using secure credentials. API keys are stored in n8n's encrypted credentials database and are not exposed to the client or written to the logs. The node uses models like GPT-4o to analyze the data and generate high-quality social media posts.

This configuration enables model temperature and parameter adjustments directly in the OpenAI node. This allows developers to tune post creativity without modifying backend code.

[Placeholder: Figure A.5: n8n Workflow]

Description: Screenshot placeholder representing the n8n visual canvas showing webhook triggers, data transformation nodes, LLM prompt nodes, and API response structures.

A.10 Security Measures

SmartPost AI implements several security measures to protect user data, secure API endpoints, and prevent abuse of the system.

The security architecture uses a defensive strategy, verifying inputs, requests, and credentials at every system boundary. This secures user and platform interactions.

A.10.1 Authentication and Authorization

User sessions are secured using JWT tokens signed with a server-side secret. The backend verifies these tokens on every protected route using custom authentication middleware. The application also implements role-based access control, checking the user's role (standard or pro) before authorizing access to advanced features.

Furthermore, JWT tokens are issued with a 7-day expiration window. This reduces the risk of session hijacking by requiring periodic user re-authentication.

A.10.2 Environment Variables

All sensitive credentials, API keys, database connection strings, and OAuth client secrets are stored in a local `.env` file. This file is ignored by git (`.gitignore`) to prevent credentials from being exposed in public repositories. In production, environment variables are managed securely through the hosting platform.

To prevent accidental secrets leakage, the server logs mask sensitive credentials, ensuring that API keys are not written to console outputs or persistent log files.

A.10.3 Token Protection

LinkedIn access and refresh tokens are encrypted before being stored in the database. This protects user tokens from exposure in the event of database access. The backend also uses state parameters during OAuth redirection flows to prevent cross-site request forgery.

The encryption process uses AES-256-GCM. This produces a unique initialization vector and authentication tag for each encrypted token, protecting tokens from lookup table attacks.

A.10.4 Input Validation

All incoming API requests are validated to prevent injection attacks and ensure data consistency. The `/api/generate` endpoint checks prompts against a banned words filter to block inappropriate requests. The system also validates email addresses during registration and limits OTP entry to 5 attempts to prevent brute-force attacks.

Validation logic uses regular expressions to verify email formats and sanitize input strings. This protects the backend from SQL injection and cross-site scripting (XSS) threats.

A.10.5 Rate Limiting and Error Management

To prevent abuse, the backend applies rate limits to sensitive endpoints. The `/api/auth/resend-code` endpoint is limited to 3 requests per hour per IP address. Error handling is designed to log technical details on the server while returning user-friendly messages to the client, preventing the exposure of internal system details.

This rate limiting uses `express-rate-limit` to track IP addresses. When a limit is hit, the server returns an HTTP 429 response, protecting the API from denial-of-service attempts.

Security Category	Threat Mitigated	Implementation Details
Password Security	Brute Force / Credential Theft	Bcrypt password hashing with 10 salt rounds
Session Management	Session Hijacking	JWT tokens with 7-day expiration and signature verification
OAuth Protection	Cross-Site Request Forgery (CSRF)	State parameters verified on callback endpoints
Credential Protection	Token Theft / DB Exposure	AES-256 token encryption for third-party access keys
API Abuse	Denial of Service / Spam	Rate limiters applied to sensitive endpoints (e.g. OTP resend)
Content Moderation	Policy Violation / Offensive Output	Banned words filter regex check on user prompts
Brute Force OTP	Account Takeover	Max 5 OTP attempts per code, 15-minute expiration window
Database Security	SQL Injection	Prisma ORM parameterized queries used exclusively
Information Disclosure	Exploit Discovery	Generic error messages returned, stack traces logged only

A.11 Testing and Validation

The application uses a multi-layered testing strategy to verify performance, validate API endpoints, and ensure reliable execution of automated workflows.

Testing procedures verify requirements across unit, integration, and functional scopes. This validates validation logic, database integration, and queue scheduling behavior.

A.11.1 Unit Testing

Unit tests focus on validating individual functions and business logic. These include testing helper functions like the password hashing and verification processes, the JWT signing and validation mechanisms, and the user prompt banned words filter. Unit tests mock external dependencies, such as database calls, to isolate code execution.

These tests verify function behavior with valid inputs, invalid parameters, and edge cases, ensuring consistent logic across updates.

A.11.2 Integration Testing

Integration tests verify the communication between different system modules. These include testing the interaction between Express controllers and the database using a test Prisma client instance. Integration tests check that transactions, like verification Purging or SocialAccount creation, succeed and roll back correctly on failure.

Integration tests run against a test MySQL database. This ensures database queries, foreign key constraints, and cascade delete actions perform correctly.

A.11.3 Functional Testing

Functional tests verify the application's end-to-end user workflows. This includes testing registration and OTP verification flows, Google sign-in routes, post creation and publication on LinkedIn, scheduling setup, and background worker executions. These tests verify the system's response to valid inputs, invalid parameters, and API failures.

Furthermore, mock environments simulate API responses from external endpoints (like LinkedIn or n8n). This allows the test suite to execute without making actual network requests.

A.11.4 Test Scenarios and Expected Results

The table below details ten test scenarios used to validate the application's features.

Test ID	Feature Tested	Test Steps	Expected Result
TS-01	Registration	Submit valid email and password	OTP sent via email, account created in verification table
TS-02	OTP Verification	Enter correct 6-digit OTP code	User status activated, temporary record deleted, JWT token returned
TS-03	OTP Limits	Enter incorrect OTP 5 times	OTP locked, client receives too many attempts error message

TS-04	Rate Limiting	Request resend OTP 4 times in an hour	HTTP 429 response, request blocked
TS-05	LinkedIn Login	Trigger OAuth flow, accept permissions	User redirected to redirect URI, tokens stored successfully
TS-06	AI generation	Submit prompt containing banned word	Prompt blocked, error message returned, no generation run
TS-07	Image generation	Submit valid prompt with image toggle	Image generated and saved to disk, local URL returned
TS-08	LinkedIn Post	Create post with text and image	Post published to LinkedIn, record saved to DB with posted status
TS-09	Job Scheduling	Create task with 2-hour interval	Worker schedules delayed job, executes task, updates execution date
TS-10	Job Failure	Trigger scheduled job without connection	Worker catches error, deactivates task, sends email notification

A.12 Deployment Process

The SmartPost AI platform is configured for deployment on Railway, a cloud hosting platform. The deployment setup includes a MySQL database instance, a Redis cache service, and app containers for the backend API and frontend client.

By utilizing containerized environments, the application builds and deploys consistently across staging and production environments, minimizing configuration issues.

A.12.1 Railway Deployment Architecture

The Railway architecture consists of four services: A MySQL database instance for persistent storage, a Redis instance for the BullMQ job queue, a Node.js API container that runs the Express server and BullMQ background worker, and a Vite static site hosting container for the React client. The services are linked using Railway's internal private networking, which secures communications between the API, database, and Redis cache.

Internal service routing uses private hostnames provided by Railway. This ensures database and Redis traffic does not pass over the public internet, protecting database connections.

A.12.2 Environment Configuration

Production environment variables are configured in the Railway dashboard. These include DATABASE_URL, REDIS_HOST, REDIS_PORT, REDIS_PASSWORD, JWT_SECRET, GOOGLE_CLIENT_ID, LINKEDIN_CLIENT_ID, LINKEDIN_CLIENT_SECRET, LINKEDIN_REDIRECT_URI, LINKEDIN_FRONTEND_URL, BREVO_API_KEY, and GOOGLE_SCRAPER_WEBHOOK_URL. These variables are injected into the container environments at runtime, separating server configuration from code.

This configuration uses variables referencing database credentials dynamically. If Railway updates the database password, the API container environment updates automatically, preventing connection drops.

A.12.3 Production Setup

The production containers are configured using Dockerfiles and a railway.json file. The backend build installs Node.js and Python dependencies, sets up the Prisma Client, and executes database migrations using `npx prisma migrate deploy`. The backend server starts by running the Express API and initializing the BullMQ worker to process scheduled tasks.

Build configurations optimize container sizing. The production build excludes development modules, reducing container start times and memory footprint.

A.12.4 CI/CD Pipeline

The platform implements continuous integration and deployment (CI/CD) via GitHub integration. When code is pushed to the main branch, Railway triggers a build automatically. The build pipeline runs unit tests, compiles the production bundles, updates database schemas, and deploys the new containers. Health checks verify service availability before routing traffic to the new deployment.

If a deployment fails the health checks, Railway cancels the update automatically, keeping the existing containers online. This deployment process prevents downtime during updates.

A.13 Sample Code Listings

This section includes key code listings from the application's implementation, showing details for the API routes, database schemas, and background job queues.

Listing A.1: Email Verification Logic (from auth.controller.js)

```
const verifyEmail = async (req, res) => {
  try {
    const { email, code } = req.body;
    if (!email || !code) {
      return res.status(400).json({ message: "Email and code are required" });
    }
    const verificationRecord = await prisma.emailVerification.findUnique({
      where: { email },
    });
    if (!verificationRecord) {
      return res.status(400).json({ message: "No pending verification found" });
    }
    if (verificationRecord.attempts >= 5) {
      return res.status(400).json({ message: "Too many failed attempts. Request a new code." });
    }
    if (new Date() > verificationRecord.expiresAt) {
      return res.status(400).json({ message: "Verification code has expired" });
    }
    if (verificationRecord.code !== code) {
      await prisma.emailVerification.update({
        where: { email },
        data: { attempts: { increment: 1 } },
      });
      return res.status(400).json({ message: "Invalid verification code" });
    }
    const newUser = await prisma.user.create({
      data: {
        email: verificationRecord.email,
        password: verificationRecord.password,
        role: "standard",
      },
    });
    await prisma.emailVerification.delete({ where: { email } });
    const token = jwt.sign(
      { userId: newUser.id.toString(), email: newUser.email, role: newUser.role },
      process.env.JWT_SECRET || "fallback_secret",
      { expiresIn: "7d" }
    );
    res.status(201).json({
      message: "Email verified successfully",
      token,
      user: { id: newUser.id.toString(), email: newUser.email, role: newUser.role }
    });
  } catch (error) {
    console.error("Verification error:", error);
    res.status(500).json({ message: "Internal server error" });
  }
};
```

Listing A.2: Token Refresh Helper (from linkedin.controller.js)

```
async function getValidToken(account) {
  const now = new Date();
  const buffer = new Date(now.getTime() + 60 * 1000); // 60s buffer
  if (account.expiresAt && account.expiresAt > buffer) {
    return account.accessToken;
  }
  if (!account.refreshToken) {
    throw new Error("LinkedIn session expired. Reconnect account.");
  }
  const params = new URLSearchParams();
  params.append("grant_type", "refresh_token");
```



```

params.append("refresh_token", account.refreshToken);
params.append("client_id", process.env.LINKEDIN_CLIENT_ID);
params.append("client_secret", process.env.LINKEDIN_CLIENT_SECRET);

const response = await axios.post(LINKEDIN_TOKEN_URL, params, {
  headers: { "Content-Type": "application/x-www-form-urlencoded" },
});
const { access_token, expires_in, refresh_token } = response.data;
const expiresAt = new Date(Date.now() + expires_in * 1000);

await prisma.socialAccount.update({
  where: { id: account.id },
  data: {
    accessToken: access_token,
    refreshToken: refresh_token || account.refreshToken,
    expiresAt,
  },
});
return access_token;
}

```

Listing A.3: Background Task Worker Process (from postQueue.js)

```

function startWorker() {
  worker = new Worker(
    QUEUE_NAME,
    async (job) => {
      const { scheduledTaskId, topic, attitude, includeImage, publishLinkedIn,
        userId } = job.data;
      console.log(`[BullMQ] Processing job ${job.id} for task ${scheduledTaskId}`);

      const youtubeData = await runPython("youtube_scraper.py", [topic, attitude]);
      const postRes = await fetch(process.env.GOOGLE_SCRAPER_WEBHOOK_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(youtubeData),
      });
      const postData = await postRes.json();

      let imageUrl = null;
      if (includeImage) {
        const imagePrompt = `${topic} - ${attitude} style, social media post image,
        vibrant, modern`;
        const imgResult = await runPython("image_generator.py", [imagePrompt]);
        if (imgResult.success && imgResult.url) imageUrl = imgResult.url;
      }

      const task = await prisma.scheduledTask.findUnique({ where: { id:
        BigInt(scheduledTaskId) } });
      if (!task) return { success: false, error: "Task not found" };

      const dbPost = await prisma.post.create({
        data: {
          socialAccountId: task.socialAccountId,
          content: postData.output,
          mediaUrl: imageUrl,
          type: includeImage ? "image" : "text",
          status: "draft",
          scheduledAt: new Date(),
        },
      });

      if (publishLinkedIn) {
        // trigger publishToLinkedInInternal and update post status to 'posted'
      }

      await prisma.researchReport.create({
        data: { postId: dbPost.id, topic, generatedReport:
          JSON.stringify(youtubeData) }
      });
    }
  );
}

```

```

    if (task.isActive) {
      const next = new Date(Date.now() + task.intervalHours * 60 * 60 * 1000);
      await prisma.scheduledTask.update({ where: { id: task.id }, data:
    { nextExecution: next } });
      await postQueue.add("generate-post", { ...job.data }, { delay:
Math.max(next.getTime() - Date.now(), 0) });
    }
    return { success: true };
  },
  { connection }
);
}

```

Listing A.4: Database Schema Definition (schema.prisma)

```

model User {
  id          BigInt          @id @default(autoincrement()) @db.UnsignedBigInt
  email       String          @unique @db.VarChar(255)
  password    String          @db.VarChar(255)
  role        Role            @default(standard)
  createdAt   DateTime        @default(now()) @db.Timestamp(0)
  socialAccounts SocialAccount[]
  drafts      Draft[]
}

model SocialAccount {
  id          BigInt          @id @default(autoincrement()) @db.UnsignedBigInt
  userId       BigInt          @db.UnsignedBigInt
  platformId   BigInt          @db.UnsignedBigInt
  accessToken  String          @db.Text
  refreshToken String?         @db.Text
  expiresAt    DateTime?       @db.DateTime(0)
  linkedinUrn  String?         @db.VarChar(100)
  linkedinName String?         @db.VarChar(255)
  user         User            @relation(fields: [userId], references: [id],
onDelete: Cascade)
  platform     Platform        @relation(fields: [platformId], references: [id],
onDelete: Cascade)
  posts        Post[]
  scheduledTasks ScheduledTask[]
  @@unique([userId, platformId], name: "unique_user_platform")
}

```

Listing A.5: YouTube Scraper Execution Script (youtube_scraper.py)

```

import sys, json, os
from googleapiclient.discovery import build

def run_scraping(prompt, attitude):
    api_key = os.environ.get("YOUTUBE_API_KEY")
    if not api_key:
        return {"success": False, "error": "YOUTUBE_API_KEY missing"}
    youtube = build('youtube', 'v3', developerKey=api_key)
    search_res = youtube.search().list(
        q=prompt, part='id,snippet', maxResults=5, type='video'
    ).execute()
    videos = []
    for item in search_res.get('items', []):
        video_id = item['id']['videoId']
        videos.append({
            "id": video_id,
            "title": item['snippet']['title'],
            "description": item['snippet']['description']
        })
    return {"success": True, "array": videos}

if __name__ == '__main__':
    prompt = sys.argv[1] if len(sys.argv) > 1 else 'AI'
    attitude = sys.argv[2] if len(sys.argv) > 2 else 'Neutral'

```

```
res = run_scraping(prompt, attitude)
print(json.dumps(res))
```

Listing A.6: Frontend Client Routes Configuration (App.jsx)

```
function App() {
  return (
    <AuthProvider>
      <ThemeProvider>
        <Router>
          <Routes>
            <Route path="/login" element={<Login />} />
            <Route path="/signup" element={<Signup />} />
            <Route path="/verify-email" element={<EmailVerify />} />
            <Route path="/" element={
              <RequireAuth>
                <AppLayout><Dashboard /></AppLayout>
              </RequireAuth>
            } />
            <Route path="/generate" element={
              <RequireAuth>
                <AppLayout><Generate /></AppLayout>
              </RequireAuth>
            } />
            <Route path="/schedule" element={
              <RequireAuth>
                <AppLayout><Scheduler /></AppLayout>
              </RequireAuth>
            } />
            <Route path="/linkedin" element={
              <RequireAuth>
                <AppLayout><LinkedIn /></AppLayout>
              </RequireAuth>
            } />
          </Routes>
        </Router>
      </ThemeProvider>
    </AuthProvider>
  );
}
```

16. Challenges & Risks

Challenge	Mitigation
Social media API rate limits	Implemented token refresh logic; use official APIs with proper quotas
Platform restrictions	Focused on LinkedIn as the primary platform; expandable architecture
AI generation costs	Pollinations AI provides free image generation; LLM costs managed via n8n webhook
Content moderation risks	Configurable banned words filter on all AI generation prompts
Ethical concerns (misinformation)	User reviews and approves every generated post before publication
Scraper reliability	Graceful degradation — scrapers return empty arrays on failure instead of crashing the pipeline
Redis dependency	Required only for scheduled posting; core features work without Redis

17. Future Improvements

- Analytics dashboard with engagement tracking (likes, shares, impressions).
- Multi-platform simultaneous posting (X/Twitter, Instagram, Facebook).
- Multi-language support for generated content.
- AI tone customization with more granular options (formal, humorous, persuasive, etc.).
- Smart trending detection — automatically suggest topics based on current trends.
- Team collaboration features with shared accounts and approval workflows.
- Advanced notification system (in-app notifications, push notifications).
- AI media improvements — video generation support.
- Admin panel for user management and platform monitoring.

18. Conclusion

SmartPost AI successfully delivers a comprehensive platform for AI-driven social media content creation. By combining web scraping, large language models, image generation, and a

robust scheduling system, it enables users to maintain an active social media presence with minimal manual effort.

The modular architecture — with separated frontend, backend, Python services, and background workers — ensures that the system is maintainable and extensible. The current LinkedIn integration demonstrates the full end-to-end workflow, and the architecture is designed to accommodate additional platforms with minimal changes.

The project demonstrates practical applications of several advanced technologies: OAuth2 authentication flows, background job processing with BullMQ, real-time web scraping, AI content generation, and responsive modern web design — all unified into a cohesive user experience.